essay · 2026-09-11

the harness is the signal

Adrian Tame Jacobo, writing for Plicara Labs · 2026-09-11 ·

The thing I find interesting about harness engineering is how familiar the problems are. We are talking about coordinating work, managing state, granting permissions and recovering when something fails. Computer science and software engineering have been dealing with these things for decades. How much of what we are calling new is actually new, and how much is us finding a different name for something we already understand?

By *harness*, I mean the runtime around the model: its tools, execution state, permissions, approvals, logging and recovery. The model decides what to try, and the harness lets that decision become an action. I'm using the term broadly because the question extends beyond any one framework or orchestration library.

There is something new in the middle of all this, though. We are building around a model that can interpret a task and decide how to approach it, but whose judgement we cannot treat as a guarantee. Some of the surrounding machinery deals with that uncertainty. Some would be necessary even if the model made every decision correctly. I think separating those two is where this gets interesting: which responsibilities need to stay outside the model, and which ones are compensating for a model that is not good enough yet?

the prompt is not the product

When an application mostly returns text, it makes sense to spend a lot of time on what you tell the model. Once it can call tools and change things in other systems, there is a lot more to account for.

OpenAI's guide to building agents describes the model as managing workflow execution through tools within defined guardrails. That description already puts a fair amount of the system outside the prompt. Say a tool changes a record, but the connection drops before it returns a result. The model now has to work out whether the action happened and whether trying again would repeat it. Better reasoning can help it decide what to check. It still needs a system that can tell it what happened. Adding another paragraph to the prompt does not create that record.

That is why I think the harness is an amazing signal. Watch what we have to put around the model to make it useful, then watch what we can remove as it improves (or add as it improves!). The

direction of that change may tell us more than the terminology we attach to it.

some of the harness is probably temporary

There is an obvious counterargument: a lot of harness engineering may just be compensation for weak models. Better models could remove some of the planning loops, self-critique and multi-agent choreography we are adding now. Anthropic's 2024 guidance on building agents starts with the simplest thing that works and adds complexity when it improves the result. OpenAI makes a similar recommendation: get as much as possible out of one agent and add tools before introducing multi-agent orchestration. That seems like a sensible place to start.

This architecture can create its own problems though. Every handoff gives us another place to lose context or misunderstand a result, and another interaction to debug. Maybe better models will need simpler, more powerful tools and less instruction about how to use them. If that happens, some of the elaborate control structures we are building today will turn out to have been unnecessary for the next generation. I would want to see that happen before calling it the answer, though. A simpler harness that fails halfway through the job is not an improvement. The comparison has to include what the system can actually finish, how reliably it does it and what it costs. A tool count alone will not tell us that.

what has to stay outside the model?

One way to sort through this is to ask what we would keep if the model got much better tomorrow. I would still want permissions enforced and a record of what it did. I might not need to make it write a plan, critique the plan and ask another agent to critique the critique. That is the distinction I mean by structural and compensatory. Here is roughly where I would put the pieces:

responsibility	likely character	why
Authentication, authorization and approvals	Structural	These are organizational and security constraints, not reasoning tasks.
Audit logs, traces and policy enforcement	Structural	Somebody outside the model needs to inspect and govern consequential actions.
Checkpoints, retries and recovery	Structural	Networks fail, rate limits happen and long-running work gets interrupted.

responsibility	likely character	why
Tool interfaces and schemas	Likely structural	The model needs a reliable way to discover and invoke capabilities outside itself.
Reflection, self-critique and repeated prompting	Possibly compensatory	Stronger models may need fewer explicit loops for some tasks.
Planner or decomposer subagents	Mixed	They can isolate or specialize work, but they can also compensate for weak execution.

These are judgements about responsibilities, not predictions that today's implementations will survive. A model might get better at deciding when to retry, for example, while the system still needs to prevent a retry from performing the same action twice. More of the decision can move into the model without the external responsibility disappearing.

This is where the older engineering work helps. Workflow systems already deal with pausing, retrying and resuming work after a process fails; Temporal's documentation describes those problems directly. Google's *Site Reliability Engineering* book deals with observing systems and defining what reliable operation means. We have a lot to draw from. The work is figuring out which of those ideas still fit when part of the program is making probabilistic judgements, and where that difference forces us to rethink them.

follow the interfaces

Model Context Protocol is interesting here less because it might win and more because of what it standardizes. It gives an LLM application a common way to connect to external tools and data sources. The interesting part is the attempt to agree on an interface. If different systems can expose capabilities in a common way, the model and its runtime have less integration work to do for each one. That does not settle how permissions should be enforced or how a failed action should be recovered. It gives those systems a boundary they can build around.

I would watch whether that agreement spreads. Are teams arriving at interfaces they can reuse, or does every new framework need its own version of everything? A specification is a useful starting point. Convergence in what people actually build would be a stronger signal.

how this could be wrong

There are two things I would be careful about here. First, a harness can get smaller because its complexity moved into a service we no longer see. Second, it can get larger because we are asking it to do harder jobs. Neither tells us much about what a better model has made unnecessary. We need to compare similar work and account for the whole system.

I would watch four things:

1. **Runtime complexity:** Do production systems get simpler as models improve, or do durable execution, evaluation and governance stay in place?
2. **Where the gains come from:** Are improvements mostly coming from the base model, or from better tools, state handling, verification and execution environments?
3. **Standardization:** Do tool and context interfaces converge across vendors and applications?
4. **Operational ownership:** Which concerns are still owned by platform, security and reliability teams when model behaviour improves?

I don't think we know which pieces of the puzzle make up the picture of AI yet. But a lot of the pieces are already on the table, from distributed systems, workflow design and control theory. We need to work out which ones apply to this way of computing, which need to change and which we can leave behind. I think this is a very strong signal as to where the wind is blowing. Are we seeing complicated harnesses win, or are we seeing simpler ones emerge? This can tell us much more about the future of these systems than trying to guess at benchmarks or anything like that.